DEPARTMENT OF CSE, CS&IT, AI&DS
COURSE CODE: 24SDCS02 / 24SDCS02E / 24SDCS02P / 24SDCS02L
FULL STACK APPLICATION DEVELOPMENT

Name: S. LALITH ADITYA ID: 2400031810

SKILL - 2 Hibernate CRUD Operations

Prerequisites:

- Basic knowledge of Java
- Understanding of classes, objects, and OOP
- Familiarity with SQL basics

A retail inventory system needs to store product details such as productId, name, description, price, and quantity. The admin should be able to add new products, retrieve product information, update price or quantity, and delete discontinued products.

- Your task is to implement complete CRUD operations using Hibernate with proper entity mapping using JPA annotations.

1. Create a Product entity with fields: id (primary key), name, description, price, quantity.
2. Configure Hibernate and map the Product entity to a table using JPA annotations.
3. Insert multiple Product records into the database.

CustomerJava

```
package com.klu.model;
import javax.persistence.*;
@Entity
@Table(name="")
public class Product {
@Id
private int id;
private String name;
private String des;
private double price;
private int quantity;
public  Product(){
}
public int getId() {
return id;
}

public void setId(int id) {
```

```
this.id =id;
}
public String getDes() {
return des;
}
public void setDes(String des) {
this.des = des;
}
public String getName() {
return name;
}
public void setName(String name) {
this.name = name;
}
public double getPrice() {
return price;
}
public void setPrice(double price) {
this.price = price;
}
public int getQuantity() {
return quantity;
}
public void setQuantity(int quantity) {
this.quantity = quantity;
}
}
```

MainApp.java

```
package com.klu.model;
import org.hibernate.Session;
import org.hibernate.Transaction;
import java.util.Scanner;
public class Main {
public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        int choice;
        do {
            System.out.println("====== Product CRUD MENU ======");
            System.out.println("1. Add Product");
            System.out.println("2. View Product");
            System.out.println("3. Update Product");
            System.out.println("4. Delete Product");
            System.out.println("5. Exit");
            System.out.print("Enter your choice: ");
         choice=sc.nextInt();
```

```
                switch(choice) {
                case 1: AddProduct(sc); break;
                case 2: ViewProduct(sc); break;
                case 3: UpdateProduct(sc); break;
                case 4: DeleteProduct(sc); break;
                case 5: System.out.println("Thank You!"); break;
                default: System.out.println("Sorry Enter correct choice");
                }
               }while(choice!=5);
        }

private static void DeleteProduct(Scanner sc) {
        Session session = HibernateUtil.getSessionFactory().openSession();
        Transaction tx4 = session.beginTransaction();
        System.out.print("Enter Student ID to Delete: ");
        int did = sc.nextInt();
        Product dst = session.get(Product.class, did); // Persistent
        if (dst != null) {
        session.delete(dst);
        tx4.commit();
        System.out.println("Product Deleted Successfully");
        } else {
        System.out.println("Product Not Found");
        tx4.rollback();
        }
        session.close();

}

private static void UpdateProduct(Scanner sc) {
        Session session=HibernateUtil.getSessionFactory().openSession();
        Transaction tx1 = session.beginTransaction();
        System.out.print("Enter Product ID: ");
        int id = sc.nextInt();

        Product st = session.get(Product.class, id);
        if (st != null) {
                sc.nextLine();
                System.out.print("Enter New Name: ");
                st.setQuantity(sc.nextInt());
                System.out.print("Enter New Marks: ");
                st.setPrice(sc.nextInt());

                // No update() needed
                tx1.commit();
                System.out.println("Student Updated Successfully");
                } else {
```

```
                System.out.println("Student Not Found");
                tx1.rollback();
                }
                session.close();
        }
private static void ViewProduct(Scanner sc) {
        Session session=HibernateUtil.getSessionFactory().openSession();
        System.out.print("Enter Product ID: ");
        int id = sc.nextInt();
        Product st = session.get(Product.class, id);
        if (st != null) {
        System.out.println("ID: " + st.getId());
        System.out.println("Name: " + st.getName());
        System.out.println("Description: " + st.getDes());
        System.out.println("Price: " + st.getPrice());
        System.out.println("Quantity: " + st.getQuantity());
        } else {
        System.out.println("Product Not Found");
        }
        session.close();
}
private static void AddProduct(Scanner sc) {
    Session session = HibernateUtil.getSessionFactory().openSession();
    Transaction tx = session.beginTransaction();
    Product p = new Product();
    System.out.print("Enter ID: ");
    p.setId(sc.nextInt());
    sc.nextLine();
    System.out.print("Enter Name: ");
    p.setName(sc.nextLine());
    System.out.print("Enter Description: ");
    p.setDes(sc.nextLine());
    System.out.print("Enter Price: ");
    p.setPrice(sc.nextDouble());
    System.out.print("Enter Quantity: ");
    p.setQuantity(sc.nextInt());
    session.save(p);
    tx.commit();
    session.close();
    System.out.println("Product Added Successfully");
}
}
```

Order.Java

```
package com.customer_order;
import javax.persistence.*;

@Entity
```

```
@Table(name="orderTable")

public class Order {

	@Id
	@GeneratedValue(strategy=GenerationType.IDENTITY)
	private int order_id;

	private String product_name;
	private double price;
	                                                                                    /*--------
-------------------------create a foreign key column ----------------------------------------------*/
	@ManyToOne
	@JoinColumn(name="ordered_customer_id")
	private Customer ordered_customer;

	/**************** constructor*****************/
	public Order() {}
	public Order(String n,double p) {
		this.product_name=n;
		this.price=p;
	}

	/************* add customer to foreign key column i.e ordered
customer;**************/
	public void setCustomer(Customer c) {
		this.ordered_customer=c;
	}

	/******************** Getters********************/
	public String getProductName() {
		return this.product_name;
	}
	public double getProductPrice() {
		return this.price;
	}
	public int getOrderId() {
		return this.order_id;
	}

}
```

main/resources/orders.cfg.xml

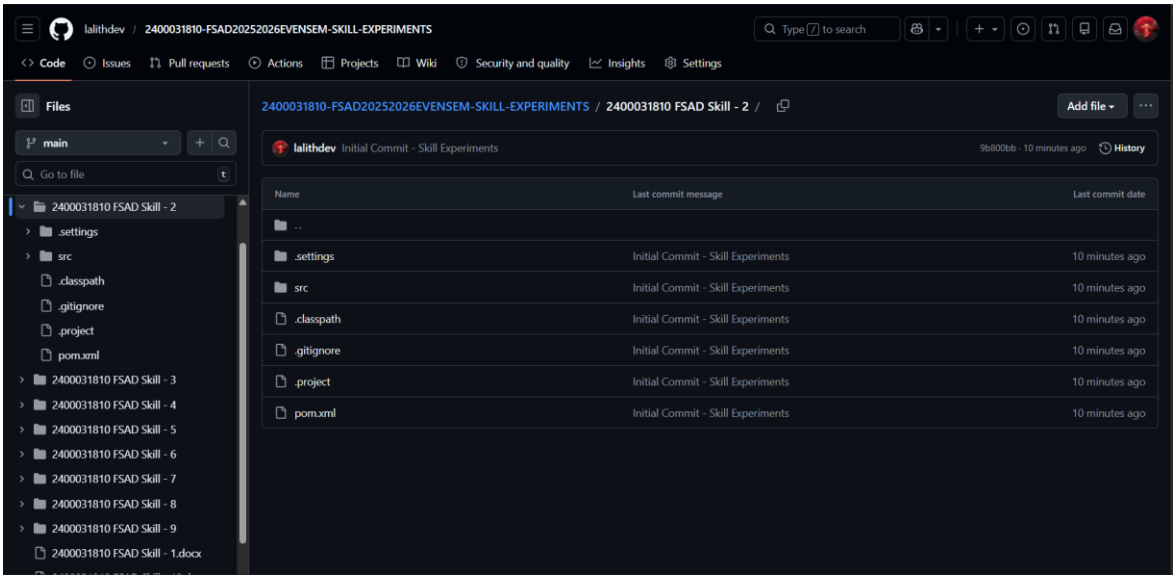
```
<hibernate-configuration>

	<session-factory>
		<property                                    name="hibernate.connection.driver_class">
	com.mysql.cj.jdbc.Driver</property>
```

```
        <property
name="hibernate.connection.url">jdbc:mysql://localhost:3306/orders</property>
        <property name="hibernate.connection.username">root</property>
        <property name="hibernate.connection.password">Praveen@06</property>

        <property name="hibernate.dialect">org.hibernate.dialect.MySQL8Dialect</property>
        <property name="hibernate.hbm2ddl.auto">update</property>
        <property name="hibernate.show_sql">true</property>

        <mapping class="com.customer_order.Order"></mapping>
        <mapping class="com.customer_order.Customer"></mapping>

    </session-factory>
</hibernate-configuration>
```

pom.xml

```
        <projectxmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        https://maven.apache.org/xsd/maven-4.0.0.xsd">
          <modelVersion>4.0.0</modelVersion>
          <groupId>com.orders</groupId>
          <artifactId>CustomerOrder</artifactId>
          <version>0.0.1-SNAPSHOT</version>
        </project>
```

4. Push the Hibernate project into a single GitHub repository.



VIVA QUESTIONS:

1. **What is ORM and why is it used in application development?**
   ORM maps Java objects to database tables, allowing developers to work with data using objects instead of SQL.
2. **Which annotation is used to mark a class as a JPA entity?**
   The @Entity annotation is used to mark a class as a JPA entity.
3. **Why must every Hibernate entity have a primary key?**
   A primary key uniquely identifies each record and helps Hibernate manage entities in memory.
4. **How do you update an existing record using Hibernate?**
   By modifying the entity object and calling session.update() or saveOrUpdate().
5. **What is the difference between get() and load() in Hibernate?**
   get() fetches data immediately and returns null if not found, while load() returns a proxy and throws an exception if not found.

(For Evaluator's use only)

Comment of the Evaluator (if Any)	Evaluator's Observation Marks Secured:_____out of _____ EMP ID & Full Name of the Evaluator: Signature of the Evaluator Date of Evaluation

Date of the Session: _/_/_ Time of The Session: _____to_____